

T2 — Matching de Produtos

Aprendizado de Máquina — Prof. Me. Otávio Parraga

Integrantes: <preencher nomes e matrículas>

Resumo. Este trabalho compara duas abordagens para o matching automático entre consultas textuais heterogêneas (ex.: FANTA LARANJA 2L C/6) e um catálogo normalizado de 14.206 produtos: (1) NLP clássico, com TF-IDF (palavras e n-gramas de caracteres) e BM25 implementado do zero; e (2) Deep Learning, com embeddings semânticos do modelo multilíngue `paraphrase-multilingual-MiniLM-L12-v2` (Sentence-Transformers), nas variantes busca densa e híbrida (BM25 top-50 → re-ranqueamento neural). No conjunto de teste (250 queries), a melhor variante clássica (TF-IDF de caracteres) atingiu **P@1 = 0,992**, MRR@5 = 0,996 e R@5 = 1,000, superando a melhor variante neural (P@1 = 0,940). A análise qualitativa mostra que o pré-processamento é decisivo para ambas as abordagens (sem ele, a neural cai de P@1 0,916 para 0,516 na validação) e que os erros remanescentes se concentram em variantes quase idênticas do mesmo produto (multipack, sabor, cor).

1. Referencial Teórico

1.1 TF-IDF e Similaridade de Cosseno

O TF-IDF (*Term Frequency–Inverse Document Frequency*) representa documentos como vetores esparsos em que o peso de cada termo cresce com sua frequência no documento (TF) e decresce com sua frequência na coleção (IDF), penalizando termos comuns e valorizando termos discriminativos [1, 2]. A similaridade entre query e documento é medida pelo cosseno do ângulo entre seus vetores, que é independente do comprimento dos textos. Utilizamos duas granularidades: **palavras** (uni e bigramas) e **n-gramas de caracteres** (3 a 5, delimitados por palavra), sendo esta última conhecida por sua robustez a erros ortográficos e variações morfológicas — características típicas das queries deste problema.

1.2 BM25

O BM25 (Okapi) é uma função de ranqueamento probabilística que estende a intuição do TF-IDF com dois refinamentos: saturação da frequência do termo (parâmetro k_1) e normalização pelo comprimento do documento (parâmetro b) [3]. É o método clássico padrão em motores de busca e costuma superar o TF-IDF em textos curtos. Implementamos o BM25 do zero (`src/bm25.py`) com índice invertido, $k_1 = 1,5$ e $b = 0,75$:

$$\text{score}(q,d) = \sum_{t \in q} \text{IDF}(t) \cdot f(t,d) \cdot (k_1 + 1) / (f(t,d) + k_1 \cdot (1 - b + b \cdot |d| / \text{avgdl}))$$

1.3 Embeddings Semânticos e Sentence-Transformers

Modelos de linguagem baseados na arquitetura Transformer [4], como o BERT [5], produzem representações contextuais densas do texto. A biblioteca Sentence-Transformers [6] ajusta esses modelos com redes siamesas para que a similaridade de cosseno entre embeddings de sentenças reflita similaridade semântica. Diferentemente do TF-IDF/BM25 (vetores esparsos, casamento lexical), embeddings capturam sinonímia e proximidade semântica mesmo sem sobreposição de palavras. Usamos o modelo multilíngue `paraphrase-multilingual-MiniLM-L12-v2` (384 dimensões, suporte a português), que roda localmente em CPU, sem custo de API.

1.4 LLMs para matching (estratégia alternativa)

Uma alternativa baseada em LLMs é o *prompting* zero-shot/few-shot: fornecer ao modelo a query e uma lista de candidatos (pré-filtrados por BM25) e pedir a escolha do mais provável. Optamos pelos embeddings por serem gratuitos, reproduzíveis e determinísticos; discutimos o uso de LLMs como melhoria futura (Seção 5).

2. Delimitação do Desenvolvimento

Implementado pelo grupo:

- Todo o pré-processamento textual (`src/preprocess.py`): normalização de unidades, tratamento de embalagens, dicionário de abreviações, stopwords;
- O algoritmo **BM25 completo**, com índice invertido (`src/bm25.py`) — sem uso de bibliotecas prontas de BM25;
- Os dois pipelines de matching e suas variantes (`src/approach1_classic.py` , `src/approach2_deep.py`), incluindo a estratégia híbrida BM25 → re-rank neural e o cache de embeddings;
- O script de avaliação com as métricas P@1, MRR@5 e R@5 (`src/evaluate.py`);
- Exploração dos dados e análises qualitativas (`src/explore.py` , `src/qualitative.py` , `src/run_no_match.py`).

Reaproveitado de bibliotecas públicas: `TfidfVectorizer` e `cosine_similarity` do scikit-learn; o modelo pré-treinado `paraphrase-multilingual-MiniLM-L12-v2` via biblioteca Sentence-Transformers; `pandas/numpy` para manipulação de dados. Nenhum código de matching foi copiado de repositórios de terceiros.

3. Metodologia

3.1 Exploração dos dados

O catálogo tem 14.206 produtos (1.833 marcas), com **29 nomes duplicados** sob ids diferentes (fonte de ambiguidade irreduzível). O arquivo `queries.csv` tem 16.441 consultas não rotuladas (mediana de 7 tokens), com forte presença de tokens de embalagem (`C/12` , `C/6` , `UND` , `12X350ML` , `CX` , `FD` — 918 ocorrências) e abreviações. Os conjuntos de validação e teste (250 queries cada) têm 100% dos `matched_id` presentes no catálogo (não há `NO_MATCH` rotulado) e estilo mais descritivo que o de `queries.csv` . Identificamos ainda famílias inteiras de queries `NO_MATCH` em `queries.csv` : marcas como Qualitá (1.234 ocorrências), Taeq, Swift, Carnilove e Mormaii não existem no catálogo.

3.2 Pré-processamento (comum às duas abordagens)

Aplicado tanto às queries quanto aos nomes do catálogo (nome + marca, quando a marca ainda não aparece no nome):

Transformação	Exemplo
Minúsculas + remoção de acentos	Guaraná → guarana

Expansão de s/ e c/	S/ACUCAR → sem acucar
Remoção de quantidades de embalagem	C/6, CX12, 6 UND → (removido)
Multipack mantém o tamanho unitário	12X350ML → 350ml
Canonização de unidades	2L, 2 litros → 2l; 15 GR → 15g
Abreviações não ambíguas	CERV → cerveja; REFRIG → refrigerante
Pontuação e stopwords (de, do, com, por...)	removidas

Duas decisões merecem destaque: **(i)** mantivemos a palavra *sem* (não a tratamos como stopwords), pois ela é discriminativa em "sem açúcar", "sem gás", "sem lactose"; **(ii)** o dicionário de abreviações é deliberadamente conservador — abreviações ambíguas como COND (condensado × condicionador) e SAB (sabão × sabonete) ficaram de fora, pois a expansão errada induz matches errados.

3.3 Pipelines

Abordagem 1 (clássica). Para cada query pré-processada, calcula-se a similaridade com todos os 14.206 produtos e retornam-se os top-5. Três variantes: TF-IDF de palavras (1–2 gramas), TF-IDF de caracteres (3–5, `char_wb`, com `sublinear_tf`) e BM25 próprio.

Abordagem 2 (deep learning). Os 14.206 produtos são codificados uma única vez pelo modelo neural (105,6 s em CPU; embeddings cacheados em disco). Duas variantes: **densa** — cosseno entre o embedding da query e todos os produtos; **híbrida** (recomendada no enunciado) — BM25 seleciona 50 candidatos e o modelo neural decide o ranking final, reduzindo o espaço de busca e o custo.

Protocolo de avaliação. Todas as decisões (escolha de variantes, ablações, hiperparâmetros) foram tomadas exclusivamente sobre `queries_val.csv`. O conjunto `queries_test.csv` foi usado uma única vez, ao final, para as métricas definitivas. Métricas: P@1, MRR@5 e R@5, conforme o enunciado.

4. Resultados e Análise

4.1 Validação (desenvolvimento) — 250 queries

Variante	P@1	MRR@5	R@5	Tempo (250 q)
TF-IDF palavras (1–2)	0,944	0,968	1,000	2,1 s
TF-IDF caracteres (3–5)	0,972	0,985	1,000	17,4 s
BM25 ($k_1=1,5$; $b=0,75$)	0,956	0,977	1,000	0,4 s
Neural densa	0,916	0,943	0,984	9,3 s
Neural híbrida (BM25 top-50 → re-rank)	0,920	0,950	0,992	8,4 s

Ablações na validação: (i) indexar só o nome, sem a marca, variou no máximo ± 1 acerto ($\pm 0,004$) — mantivemos nome+marca; (ii) **sem pré-processamento**, a abordagem neural despenca: densa $P@1 = 0,516$ e híbrida $0,592$ (vs. $0,916/0,920$) — ou seja, a normalização de abreviações e unidades é crítica mesmo para modelos semânticos, que não reconhecem jargões de pedido como *C/6* ou *CERV*.

Com base na validação, selecionamos **TF-IDF de caracteres** como variante final da Abordagem 1 e **híbrida** como variante final da Abordagem 2.

4.2 Teste (métricas definitivas) — 250 queries

Abordagem	P@1	MRR@5	R@5	Tempo (250 q)	Custo	Complexidade
Ab. 1 — TF-IDF caracteres	0,992	0,996	1,000	14,0 s	Gratuito (CPU)	Baixa
Ab. 1 — BM25 (própria)	0,976	0,987	1,000	0,3 s	Gratuito (CPU)	Baixa
Ab. 1 — TF-IDF palavras	0,920	0,952	0,996	1,6 s	Gratuito (CPU)	Baixa
Ab. 2 — Neural densa	0,940	0,960	0,984	10,1 s	Gratuito (CPU local)	Média
Ab. 2 — Neural híbrida	0,940	0,960	0,984	8,9 s *	Gratuito (CPU local)	Média

* mais 105,6 s de indexação única do catálogo (amortizada; o cache em disco elimina esse custo nas execuções seguintes).

4.3 Análise qualitativa

Acertos. Ambas as abordagens resolvem com folga queries descritivas com a mesma base lexical do catálogo, mesmo com ordem de palavras diferente ("*Mortadela Fatiada Defumada Sadia Soltíssimo 200g*" → "*Mortadela defumada fatiada soltíssimo sadia 200g*") e queries abreviadas após normalização ("*CORONA CERO SUNBREW LONG NECK 330ML - C/24*" é resolvida pelo TF-IDF-char).

Erros da Abordagem 1 (apenas 2 no teste). Ambos são variantes quase idênticas do produto correto: "*Refrigerante Laranja Fanta Lata 350ml*" → previu a versão "*...lata 350ml 12 un*" (empate técnico, $\Delta\text{score} = 0,0000$; o correto ficou em 2º); e "*Espumante Casa Valduga Arte Tradicional Brut 750ml*" → previu brut **rosé** em vez de brut **branco** (a query não especifica a cor — caso genuinamente ambíguo).

Erros da Abordagem 2 (15 no teste), em três padrões:

- **Variante/sabor errado** (9 casos): "*Refresco em Pó Maracujá Tang*" → "...manga Tang"; "*Colgate Total 12 Antitártaro*" → "...gengiva reforçada". Embeddings aproximam sabores/variantes semanticamente próximos, perdendo o detalhe lexical que decide o match;
- **Categoria errada por proximidade semântica** (3 casos): "*Pipoca para Microondas YOKI*" → "*Saco hermético para freezer e microondas*" — o termo dominante no embedding ("microondas") arrasta o match para outra categoria;
- **Tamanho/quantidade errada** (3 casos): "*Salsicha Viena Ceratti 500g*" → "*Salsicha Viena Perdigão 500g*" (priorizou o peso, errou a marca).

Comparação direta (teste): em 14 queries o clássico acertou onde o neural errou; em apenas 1 o neural acertou onde o clássico errou (a Fanta multipack, por desempate fortuito); 1 erro em comum (Casa Valduga, ambíguo). Não identificamos um "tipo" de query em que a abordagem neural seja sistematicamente melhor neste dataset — o vocabulário do catálogo é controlado e, após a normalização, o casamento lexical é quase sempre suficiente. A vantagem teórica dos embeddings (sinonímia: "*refrigerante de cola*" ≈ "*coca-cola*") raramente é exigida pelas queries avaliadas.

Casos ambíguos. O catálogo tem 29 nomes duplicados sob ids distintos (ex.: "*Adoçante líquido sucralose linea 75ml*" com dois EANs) — nesses casos nenhum sistema distingue os ids apenas pelo texto. No teste, os empates relevantes são versões unitárias × multipack ("*...lata 350ml*" × "*...lata 350ml 12 un*", Δscore = 0,0000) e variantes de cor não especificadas na query (brut branco × rosé).

Casos NO_MATCH. Val/teste não contém NO_MATCH, mas `queries.csv` contém famílias inteiras (Qualitá, Taeq, Swift, Carnilove, Mormaii...). Rodamos as duas abordagens nessas queries (`results/no_match.md`): nenhuma tem mecanismo nativo de rejeição — ambas sempre devolvem algum produto. A diferença está nos scores: no TF-IDF-char, o top-1 de queries NO_MATCH fica entre 0,24 e 0,68, bem abaixo do p5 dos matches corretos na validação (0,891) — um limiar de rejeição em ≈0,80 separa quase perfeitamente os dois regimes. Já na neural os scores continuam altos em NO_MATCH (até 0,90 para o limpador Qualitá, que tem substitutos semanticamente próximos), tornando o limiar pouco confiável. Para produção, recomendamos o limiar sobre o score do TF-IDF-char, marcando como NO_MATCH e enviando para revisão humana tudo abaixo dele.

4.4 Tradeoff simplicidade × desempenho

Neste problema, o método mais simples é também o mais preciso: o TF-IDF-char vence em P@1 e o BM25 entrega 0,976 com custo computacional ~40× menor (0,3 s vs. 8,9 s + indexação) e zero dependências pesadas. A abordagem neural agrega complexidade (download de modelo de 470 MB, PyTorch, indexação) sem ganho de acurácia aqui — mas seria a escolha certa se as queries usassem vocabulário livre/sinônimos distantes do catálogo, ou em cenários multilíngues. A variante híbrida confirma a recomendação do enunciado: re-ranquear só os top-50 do BM25 melhora a R@5 da neural (0,992 vs. 0,984 na validação) e reduz o tempo.

5. Conclusão, Melhorias e Dificuldades

Conclusões. (1) Em catálogos com vocabulário controlado, métodos clássicos bem pré-processados são extremamente competitivos — TF-IDF de caracteres atingiu $P@1 = 0,992$ no teste. (2) O pré-processamento domina o resultado em ambas as abordagens: é ele que traduz o jargão dos pedidos (C/6, 12X350ML, CERV) para o vocabulário do catálogo; sem ele, a neural perde 40 pontos de $P@1$. (3) Os erros restantes não são de "entendimento", mas de granularidade: multipack, sabor e cor — atributos que exigem regras de negócio (ex.: extrair e comparar quantidade/sabor como campos estruturados), não modelos maiores.

Melhorias futuras. Fine-tuning do bi-encoder com pares positivos minerados de `queries.csv` (triplet loss); re-ranqueamento com cross-encoder ou LLM (zero/few-shot sobre os top-10 do BM25), que tende a resolver os empates de variante; extração estruturada de atributos (quantidade, sabor, multipack) como filtro pós-ranking; calibração formal do limiar de NO_MATCH com um conjunto rotulado de rejeição; fusão de scores (BM25 + embedding) em vez de cascata.

Dificuldades. Abreviações ambíguas (COND, SAB) que não podem ser expandidas com segurança; variantes multipack idênticas no texto unitário; nomes duplicados no catálogo; e, no ambiente Windows, conflito de DLL entre pandas e PyTorch (resolvido fixando a ordem de import — documentado no README).

6. Referências

1. SALTON, G.; BUCKLEY, C. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, v. 24, n. 5, p. 513–523, 1988.
2. MANNING, C. D.; RAGHAVAN, P.; SCHÜTZE, H. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
3. ROBERTSON, S.; ZARAGOZA, H. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, v. 3, n. 4, p. 333–389, 2009.
4. VASWANI, A. et al. Attention is all you need. In: *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
5. DEVLIN, J. et al. BERT: Pre-training of deep bidirectional transformers for language understanding. In: *Proceedings of NAACL-HLT*, 2019.
6. REIMERS, N.; GUREVYCH, I. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In: *Proceedings of EMNLP-IJCNLP*, 2019.
7. PEDREGOSA, F. et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, v. 12, p. 2825–2830, 2011.